

NSL Extension Directions: Deep Research into Five Uncovered Technique Domains

Status: Research paper (extension directions; survey of external state-of-the-art plus NSL-specific design sketches) **Date:** 2026-04-16 **Audience:** Engineers planning NSL's next research phase; reviewers assessing coverage gaps; contributors looking for a new technique to own

Abstract

The fourteen shipped NSL technique papers cover precision (CFTP, CFIP, M35 FP8 MMA, CTQS), attention (CSHA, CFA, PCA), adapters (WRGA), memory (FASE), export (@export), distributed training (CPDT), distillation (CPKD), adaptive optimization (CNAG), and fine-tuning (CPFT). This paper identifies five technique domains that are industry-standard but uncovered by any NSL paper: (1) KV cache quantization, (2) gradient compression for all-reduce, (3) activation recomputation as a first-class technique, (4) structured sparsity, and (5) tokenizer/embedding/head-layer-specific quantization.

For each gap, this paper surveys the 2024–2026 state of the art in external research, identifies why NSL has not addressed the gap to date, describes what an NSL-specific realization would look like that leverages NSL's unique compiler and type-system advantages, and sketches the implementation scope. The five gaps are then prioritized by impact, dependencies, and implementation cost. Two gaps (KV cache quantization, activation recomputation) are recommended as the next research priorities; the remaining three are staged for follow-on work.

§1 Introduction

§1.1 Why these five gaps

The five gaps were identified by a systematic cross-paper audit in April 2026. Each domain meets two criteria: (1) a significant industry or research consensus that the technique matters for modern ML systems (training, inference, or both), and (2) no NSL paper addresses it, even in passing. The audit considered approximately thirty candidate domains and rejected twenty-five that were either covered by existing papers (e.g., mixed precision is CFTP+CTQS; efficient attention is CSHA+CFA+PCA) or were sufficiently niche that NSL need not prioritize them (e.g., federated learning, lottery-ticket pruning).

The remaining five are the ones where NSL's lack of coverage is a legitimate gap, not a deliberate scoping decision.

§1.2 Organization

§2 covers KV cache quantization. §3 covers gradient compression for distributed training. §4 covers activation recomputation. §5 covers structured sparsity. §6 covers tokenizer/embedding/head-layer quantization. §7 prioritizes and sequences the gaps.

Each gap section has the same subsection structure: §.1 *external state of the art*, §.2 why NSL has not addressed this, §.3 *NSL-specific realization*, §.4 implementation scope.

§2 Gap 1 — KV cache quantization

§2.1 External state of the art

The KV cache — the tensor of keys and values retained across autoregressive decoding steps — dominates inference memory at long contexts. For LLaMA-70B at 128K tokens the KV cache consumes tens of gigabytes of HBM, often exceeding the weight footprint. For a 1M-token context on LLaMA-70B the cache is on the order of 328 GB, which a single H100 cannot hold.

Four major research directions have emerged:

KVQuant (Berkeley, NeurIPS 2024) pioneered sensitivity-aware KV cache quantization. It combines per-channel, pre-RoPE key quantization with non-uniform quantization (NUQ), dense-and-sparse quantization for outlier handling, and attention-sink-aware special-casing for the first tokens. The technique enables LLaMA-7B inference at 1M tokens on a single A100 and 10M tokens on an 8-GPU system — a factor-of-10 extension of usable context at fixed hardware.

KIVI (ICML 2024) is a 2-bit asymmetric scheme, per-channel for keys and per-token for values. It is simpler than KVQuant and achieves approximately $2.6\times$ memory reduction with minimal accuracy loss. KIVI is the standard open-source baseline.

TurboQuant (Google Research, ICLR 2026) combines PolarQuant (conversion to polar coordinates for natural distribution shaping) with Quantized Johnson-Lindenstrauss (QJL) for residual compression. It achieves 3.5-bit compression with near-zero accuracy loss on LongBench and Needle-in-a-Haystack, requires no calibration data, and is training-free. It outperforms KIVI at equivalent bit budgets. The training-free property is practically important: KVQuant and KIVI both require calibration data, whereas TurboQuant derives quantization parameters from probability theory.

NVFP4 KV cache (NVIDIA, 2026) targets the Blackwell architecture specifically. The NVFP4 format uses E2M1 floating point with FP8 E4M3 scaling factors and block granularity of 16 elements. On Llama 3.3 70B it achieves under 1% accuracy loss with approximately 50% memory reduction and 5% higher accuracy than MXFP4 on MMLU. Blackwell's fifth-generation Tensor Cores have native NVFP4 support.

The orthogonal technique is **PagedAttention** (vLLM), which manages KV cache memory in fixed-size blocks rather than per-sequence contiguous allocations. PagedAttention does not reduce KV memory per token; it eliminates fragmentation. It composes with any of the quantization schemes above.

§2.2 Why NSL has not addressed this

NSL has PCA (paged KV cache) from the m25 examples — this is the PagedAttention-equivalent allocation management. But PCA is about *where* KV cache lives, not *how precisely* it is stored. No NSL paper addresses the precision of the KV cache.

The likely reason is that NSL's initial research focus was on training: the papers on CFTP, FASE, CSHA, WRGA, and CPDT all address training-time concerns. Inference deployment has been a secondary priority, and within inference, the focus has been on weight and activation precision (CTQS, CFIP) rather than KV cache precision. This is a reasonable scoping choice that has left the gap open.

§2.3 NSL-specific realization: CKVQ (Compiler-Known Value Quantization)

An NSL-specific KV cache quantization technique — call it **CKVQ** for Compiler-Known Value Quantization — has three design properties that no external technique has:

(1) Compile-time layer sensitivity analysis. KVTuner and similar methods pick per-layer bit budgets from calibration data at runtime. NSL's type system knows layer structure at compile time. A CKVQ analysis pass can read the model's attention layer graph, identify which layers have RoPE, which use grouped-query attention, and which are at position-of-attention-sink (typically layer 0 and the first few tokens) — all without data. Combined with GEBA's symbolic bound mechanism, CKVQ could assign bit budgets *before* the first calibration datapoint arrives. Calibration would then refine, not originate, the per-layer choice.

(2) Compiler-emitted pre-RoPE quantization. KVQuant's key insight is that keys have different outlier structure before and after RoPE rotation — quantizing before RoPE captures outliers in aligned channels, which permits much lower bit budgets. In PyTorch this requires manual instrumentation; in NSL, the RoPE application point is visible to the compiler (from the @rope annotation and the MIR kernel structure). CKVQ would automatically identify the pre-RoPE point and emit the quantization there, with no user-level API change.

(3) Integration with PCA paged allocation. NSL's PCA allocates KV in pages; CKVQ would quantize at page granularity. Each page carries a per-page scale factor. This composes naturally with the page's fixed-size block structure and avoids the fragmentation that per-token scaling introduces. The page becomes the natural unit of quantization.

The format itself would be a mix of existing NSL formats and new ones: E4M3 for pre-RoPE keys (leveraging M35), IM3-style integer for post-attention-sink values, and a FP8 variant with a per-page scale for the general case. The attention kernel (emitted by CFA) needs a small change: the dequant step now reads a per-page scale from the page metadata.

§2.4 Implementation scope

- **Analysis pass.** A new WGGO sub-pass that reads attention layer metadata and assigns per-layer KV cache formats. Output feeds into CFIP's format table. ~400 LOC in Rust.
- **PCA integration.** Each page gains a scale factor stored in page metadata. ~200 LOC in the PCA implementation.
- **CFA emitter extension.** The attention kernel dequants keys and values before use. The dequant uses the page's scale factor. ~300 LOC of PTX emitter logic in CFA.
- **Runtime support.** The KV cache allocator respects per-page scale fields. ~150 LOC in the runtime.
- **Calibration integration.** An optional calibration step that refines the symbolic per-layer bit assignments with data. ~250 LOC.
- **Testing.** Numerical equivalence between CKVQ and full-precision at declared accuracy targets, page-scale correctness tests, RoPE-application-point tests. ~600 LOC of tests.
- **Total.** Roughly 1900 LOC across five files, plus tests. Estimated 8-12 implementation sessions. Interacts with PCA, CFA, CFIP, and WGGO.

Dependencies: PCA (shipped), CFA (shipped), CFIP (shipped), M35 (shipped). No new dependencies.

§3 Gap 2 — Gradient compression for all-reduce

§3.1 External state of the art

In data-parallel distributed training, gradient all-reduce is often the bottleneck. At multi-node scale, the inter-node network bandwidth (even with InfiniBand or NVLink) is orders of magnitude slower than intra-node HBM. Gradient compression reduces the bytes transmitted per all-reduce.

PowerSGD (Vogels et al., 2019) approximates the gradient matrix with a low-rank factorization computed via power iteration. The low-rank factors P and Q are all-reduce-compatible (the sum of factorizations is a factorization of the sum, approximately). Error feedback preserves convergence. PowerSGD compresses gradients by 120× in practice and reduces communication time by 54% on ResNet-18 CIFAR10 and 90% on an LSTM on WikiText-2. PyTorch includes a PowerSGD communication hook. Notable: a follow-up 2025 paper (PowerSGD+) identified a non-convergence counterexample in the original and proposed periodic SVD reprojection as a fix.

QSGD (Alistarh et al., 2017) applies stochastic quantization to gradients: each element is probabilistically rounded to one of two neighboring quantization levels. QSGD is the canonical gradient quantization baseline; it achieves 4× to 32× compression with bounded variance increase.

signSGD takes QSGD to 1-bit: only the sign of each gradient is transmitted, with majority voting for aggregation. It is not all-reduce-compatible (sign majority is not associative); it requires all-gather. This limits scaling.

TopK / DGC (Deep Gradient Compression) sends only the top-K elements by magnitude. Like signSGD, it is not all-reduce-compatible. TopK with DGC achieves up to 5000× compression on LSTM; at that compression rate, DGC shows severe performance degradation on small models.

LQ-SGD (2025) combines PowerSGD's low-rank factorization with logarithmic quantization of the factors P and Q. This adds a $32/b$ compression factor on top of PowerSGD's rank-based compression.

1-bit Adam (Tang et al., 2021) quantizes the variance-normalized gradient to 1 bit. It is a specific integration with the Adam optimizer.

The trade-off axes are: (a) compression ratio, (b) all-reduce compatibility (associativity), (c) convergence impact, and (d) computational overhead of compression/decompression. No single method dominates; the right choice depends on network speed, batch size, model size, and optimizer.

§3.2 Why NSL has not addressed this

CPDT plans sharding and all-reduce but treats the all-reduce as a fixed primitive: take gradients, sum across ranks, divide by world size. The all-reduce operation itself is not configurable. Gradient compression would require CPDT to emit a different all-reduce primitive (or a precomputation step before the standard all-reduce) based on compile-time-chosen compression algorithm.

This has not been done because, to date, NSL's distributed training has been validated at single-node scale with NVLink, where intra-node bandwidth is high enough that gradient compression provides minimal benefit. At that scale, the compression overhead can exceed the bandwidth saving. At multi-node scale with inter-node networking, the benefit inverts and compression becomes important. NSL has not yet been validated at that scale.

§3.3 NSL-specific realization: CGAC (Compiler-Generated All-Reduce Compression)

CGAC would extend CPDT with compile-time gradient compression planning. Three design properties distinguish it from external implementations:

(1) Per-layer compression format chosen at compile time. PyTorch's PowerSGD hook applies one algorithm globally. NSL's CPDT could choose, per layer, between PowerSGD (for large matrix-shaped gradients where low-rank approximation is lossless), QSGD (for small tensors), 1-bit (for highly redundant gradients), or uncompressed (when network is fast enough). This per-layer decision is made at WGGGO time based on the layer's tensor shape, gradient statistics (from a few training steps), and the measured network bandwidth.

(2) Error feedback is compiler-managed. PowerSGD's error feedback requires a per-tensor buffer that persists across steps. In PyTorch this is user-managed; in NSL, the buffer is allocated by the runtime, tracked by the compiler, and freed when training ends. This eliminates a common bug class (error feedback state not reset correctly between runs).

(3) Composition with FASE Deferred. When FASE is Deferred (no gradient buffer; gradients accumulate in MHat/VHat), the all-reduce target changes. CGAC would compress MHat's gradient contribution rather than a separate gradient buffer. This is a direct composition with FASE that no external technique has addressed because no external technique has FASE.

(4) Composition with CFIP. When backward kernels produce gradients already in CFIP-quantized form (IL2 or IM3), CGAC could skip the compression step entirely if the format is already compact enough. Alternatively, CGAC could use the CFIP format as the *transmission format*, avoiding a dequantize-compress round-trip. This composition is specifically enabled by having both techniques in the same compiler.

Format menu: PowerSGD with rank 1, 2, 4, or 8; QSGD with 2, 4, or 8 bits; 1-bit with error feedback; uncompressed (baseline).

§3.4 Implementation scope

- **CPDT plan extension.** WGGO gains a per-layer "compression algorithm" decision. ~500 LOC.
- **PowerSGD kernel.** A power-iteration primitive + QR decomposition for the low-rank factorization. This is linear algebra; Cranelift IR can compile it for CPU, and a dedicated CUDA kernel for GPU. ~800 LOC including both paths.
- **QSGD kernel.** Stochastic quantization with all-reduce. ~300 LOC.
- **Error feedback buffer management.** Runtime allocation, tracking, reset. ~250 LOC.
- **FASE integration.** Target selection based on FASE mode. ~150 LOC.
- **CFIP format passthrough.** When gradient is already in a compatible CFIP format, use it directly. ~200 LOC.
- **Testing.** Convergence parity with uncompressed training on small benchmarks, compression-ratio verification, error-feedback-state persistence, multi-node integration. ~1000 LOC.
- **Total.** Roughly 3200 LOC. Estimated 12–18 implementation sessions. Depends on a multi-node test environment (currently NSL's test matrix is single-node).

Critical dependency: multi-node test infrastructure. This is the reason to stage CGAC after CKVQ: CKVQ can be validated on single-node; CGAC cannot.

§4 Gap 3 — Activation recomputation as a first-class technique

§4.1 External state of the art

Activation recomputation (also called gradient checkpointing) trades compute for memory. During the forward pass, certain activations are not saved; during the backward pass, they are recomputed from the preceding saved activation. The standard trade-off is 30% extra compute for 2–5× activation memory reduction.

Full recomputation (Chen et al., 2016) saves only the inputs to each transformer layer and recomputes everything within. This is the coarsest granularity and the highest compute overhead (~30%).

Selective activation recomputation (Korthikanti et al., Megatron) recognizes that different activations have different recompute costs. Self-attention intermediate outputs (pre-softmax scores, softmax output) take sequence-length-squared memory but require cheap computation to reproduce — so they are the ideal checkpointing targets. MLP activations are cheap to store but expensive to recompute, so they should be kept in memory. Megatron's selective recompute uses this heuristic to achieve ~5× activation memory reduction at only ~5% compute overhead.

Megatron Bridge / NeMo exposes fine-grained selective modes:

`recompute_granularity="selective"` with `recompute_modules=["core_attn", "mlp", "layernorm", "moe", "moe_act", "shared_experts", "mla_up_proj"]`. Users pick which modules to recompute.

Adacc (2025) uses a MILP solver to jointly decide, per-tensor, whether to (a) save in full precision, (b) save in compressed form (INT4 with outlier-separated compression), or (c) recompute. It also introduces an adaptive policy evolution mechanism that adjusts decisions during training as gradient statistics change. Adacc reports 1.01× speedup — small but positive — along with substantial memory savings.

PyTorch 2.4+ activation memory budget

`(torch._functorch.config.activation_memory_budget)` allows the user to set a target activation memory usage as a fraction of default; PyTorch's inductor then runs an automatic checkpointing solver. The implementation is imperfect (may not account for recomputation memory overhead; known issue) but it is the most user-friendly production API.

TorchTitan (ICLR 2025) composes selective activation checkpointing with FSDP, tensor parallelism, pipeline parallelism, and torch.compile. It is the current open-source reference implementation for large-scale training.

§4.2 Why NSL has not addressed this

CSHA Tier C is the only NSL-shipped technique that resembles activation recomputation: Tier C saves Q_proj/K_proj/V_proj/row_max/row_sum, and the backward kernel reads them rather than recomputing from the attention-layer inputs. This is a specific, attention-kernel-internal save-activation contract; it is not a general recomputation framework.

The "@checkpoint" annotation was listed as a deferred follow-up in Tier C's TODO list. No further work has landed. The composition paper (§3.3) notes this explicitly.

NSL's FASE addresses gradient memory (the gradient buffer). Activation memory is the other major training-time memory consumer, and it is larger than the gradient buffer at medium batch sizes (as shown by Adacc's study: 66–76% of memory is activations).

§4.3 NSL-specific realization: CCR (Compiler-Chosen Recomputation)

CCR would treat recomputation as a first-class compiler decision. Four design properties:

(1) MILP-solved recomputation plan, run at WGGO time. Adacc's approach — MILP for per-tensor save-vs-compress-vs-recompute — is a compile-time decision, not a runtime one. NSL already has the compile-time symbolic analysis infrastructure (GEBA, WGGO) that makes MILP solving tractable. A CCR pass would build the memory graph from MIR, pose the MILP, and produce a per-activation-tensor decision table. This table feeds into the runtime's memory planner (M36).

(2) Adaptive policy evolution via CTQS-style phase transitions. Adacc's adaptive policy is a runtime adjustment. CTQS already has phase-transition machinery (loss-based, step-count-based). CCR would hook into CTQS's phase system: a new phase can revise the recomputation plan. This gives NSL the adaptive capability without a new phase-transition mechanism.

(3) Tight integration with CSHA and CFIP. Tier C save-activations and CCR's general recomputation are two points on the same design space. A unified view: CSHA tiers A/B/C + CCR recompute = five points on the "save X in memory, compute Y on demand" spectrum. The unified view lets CCR's decision table specify the CSHA tier for attention layers and the recomputation decision for MLP layers in the same framework. Additionally, when CCR decides to save an activation, CFIP can decide to save it in a compressed format (IF4, IL2) rather than full precision — Adacc's compression-vs-recomputation dichotomy becomes compression-with-CFIP-or-recomputation-or-both.

(4) Composition with FASE and CPFT. A frozen layer (CPFT) whose activations are not used in backward does not need its activations saved or recomputed — the backward simply does not need them. CCR's MILP reads CPFT's per-layer trainability status to eliminate unnecessary decisions. FASE's Deferred mode does not affect activation memory directly, but its gradient-buffer savings are not redundant with CCR's activation savings — both contribute independently.

The user-visible API would be a `@checkpoint(budget="auto")` or `@checkpoint(budget=0.8)` annotation, matching PyTorch's memory budget semantics.

§4.4 Implementation scope

- **CCR analysis pass.** Builds the activation memory graph from MIR, solves the MILP, outputs per-tensor decisions. MILP solver: use `lp_solve` or an integer programming crate. ~700 LOC.
- **M36 memory planner integration.** The planner reads CCR's decision table and allocates accordingly. ~300 LOC.
- **Recompute execution runtime.** Recomputed activations are produced on-demand during backward. ~400 LOC.
- **CSHA Tier unification.** CSHA's tier selection is folded into CCR's framework. Existing CSHA code is refactored, not rewritten. ~200 LOC.
- **CFIP compressed-save integration.** When CCR decides to save an activation, it can save in a compressed CFIP format. ~150 LOC.
- **CPFT integration.** Frozen layers skip activation decisions. ~100 LOC.
- **Phase-transition integration (CTQS).** A new CTQS phase can trigger CCR re-planning. ~150 LOC.
- **Testing.** End-to-end memory-budget compliance, numerical equivalence, composition with FASE and CPFT. ~800 LOC.
- **Total.** Roughly 2800 LOC. Estimated 10-14 implementation sessions. Depends on CSHA, CFIP, WGGO, M36 (all shipped).

This is the most obviously-useful of the five gaps because activation memory is the single biggest training memory consumer and every other framework has this feature.

§5 Gap 4 — Structured sparsity

§5.1 External state of the art

Fine-grained structured sparsity uses a specific zero pattern that hardware can accelerate. The canonical pattern is **2:4**: in every group of four contiguous values, at least two must be zero. This pattern:

- halves the memory footprint of one operand (the one with 50% sparsity)
- enables the hardware to skip the zero values, potentially doubling matmul throughput
- is structured enough to retain after post-training pruning with modest accuracy loss

NVIDIA Ampere (A100) introduced 2:4 sparse tensor cores. Theoretical 2× throughput gain. Practically, the speedup ranges from 1.27× to 1.6× on typical LLM inference (the gap between theoretical and practical comes from memory-bound regimes, non-sparse-amenable dimensions, and CUTLASS kernel maturity).

NVIDIA Hopper (H100) extends 2:4 support to FP8. Sparse FP8 kernels via CUTLASS achieve $\sim 1.7\times$ lower latency and $1.5\times$ higher throughput than dense FP8 on Llama 3.1 8B (Red Hat Neural Magic, 2024). Full accuracy recovery with two-part quantization strategy (dynamic per-token FP8 activations + static per-channel FP8 weights).

NVIDIA Blackwell (B200) introduced **4:8 pair-wise structured sparsity for NVFP4**: in every group of eight elements, two pairs must be zero. This preserves the 50% sparsity rate but aligns with Blackwell's fifth-generation Tensor Core granularity.

PyTorch semi-structured sparsity provides `SemiSparseLinear` and sparsity-aware training. DINOv2 ViT-L on $4\times$ A100 achieves 6% wall-time reduction for training with virtually no accuracy loss (82.8 $\rightarrow$ 82.7 on ImageNet top-1). For inference, segment-anything achieves 10% end-to-end speedup.

Accuracy recovery. 2:4 pruning without fine-tuning loses 1–3 points on LLM benchmarks. With sparse fine-tuning (SparseGPT, Wanda) the loss drops below 1 point for most models. Tencent's progressive training recipes plus quantization achieve $1.3\text{--}1.8\times$ end-to-end acceleration at minimal accuracy cost.

Industry adoption is limited. Despite the theoretical $2\times$ throughput, most production inference (OpenAI, Anthropic, Google) does not use 2:4 sparsity. Reasons: accuracy recovery difficulty, kernel maturity gaps (CUTLASS is behind cuBLAS in optimization), TDP-limited workloads where compute savings don't translate to wall-clock savings, and the operational cost of maintaining a sparse-model pipeline alongside dense. Meta experiments with it in Llama; Chinese AI labs use it more.

§5.2 Why NSL has not addressed this

NSL's type system has `Sparse<[shape], dtype, format>` in `types.rs`. This is scaffolding; there is no paper describing how sparsity patterns are chosen, how pruning is scheduled during training, how pruned kernels are emitted, or how sparsity composes with the other techniques.

The scaffolding was likely put in place with the expectation that a future paper would develop it. That paper has not been written.

§5.3 NSL-specific realization: CSS (Compiler-Scheduled Sparsity)

CSS would treat structured sparsity as a compile-time scheduling decision. Four design properties:

(1) Compile-time pruning-schedule synthesis. The user declares a target sparsity pattern (e.g., `@sparsity(pattern="2:4")`) and a recovery budget (e.g., `recovery_epochs=10`). CSS generates a progressive training recipe: start dense, gradually mask the bottom-2-of-4 weights per group, decrease the density schedule, retrain to recover accuracy. The recipe is synthesized at compile time from the hardware target and the recovery budget.

(2) Native sparse-tensor type. `Sparse<[M, N], f16, "2:4">` is a first-class type. The type carries (a) the pattern format, (b) the non-zero value storage, and (c) the metadata (4-bit indices per 4-element group). Operations on `Sparse<...>` types emit sparse CUTLASS kernels; operations between a `Sparse<...>` and a dense tensor emit mixed kernels; operations between two `Sparse<...>` tensors require compatible patterns. The type system catches incompatibilities at compile time.

(3) Composition with CFTP and M35. 2:4 sparse FP8 (Hopper) and 4:8 sparse NVFP4 (Blackwell) are the high-throughput paths. CSS chooses sparsity patterns compatible with the target hardware's FP8/NVFP4 support, leveraging M35's existing format handling.

(4) Composition with WRGA adapters. Low-rank adapters don't naturally admit 2:4 sparsity (the rank-r intermediate has no 4-element structure). But the base matrix can be sparse while the adapter is dense. CSS would handle this composition at the WRGA kernel level: the base MMA uses the sparse path, the adapter uses the dense path, and the result is summed. This is a unique composition enabled by NSL having both techniques.

(5) Composition with CFIP. A 2:4-sparse tensor's non-zero values can still be CFIP-format-assigned. The sparsity operates at the structural level; CFIP operates at the precision level. They are orthogonal and compose: a 2:4 sparse IM3 tensor has half the elements of dense, each at 3 bits. This is the high-compression corner of the design space.

§5.4 Implementation scope

- **Sparse type lowering.** Semantic stage recognizes `Sparse<...>` types and lowers to tensor + metadata pair. ~300 LOC.
- **Progressive pruning schedule synthesis.** Compile-time generator producing a runtime schedule of mask updates and density decreases. ~400 LOC.
- **Sparse PTX emitter (2:4).** Use CUTLASS sparse kernel templates; emit calls with appropriate scale factors and dtype. ~500 LOC.
- **Sparse PTX emitter (4:8 for NVFP4).** Target Blackwell; uses fifth-gen tensor core sparse instructions. ~350 LOC.
- **WRGA composition.** Base matrix + adapter kernel for sparse base. ~300 LOC.
- **CFIP composition.** Interior formats applied to non-zero entries of sparse tensors. ~200 LOC.
- **Runtime mask update.** During progressive pruning, update the 2:4 mask at prescribed step boundaries. ~250 LOC.
- **Testing.** Accuracy recovery on a small LLM, throughput benchmarks vs dense, composition with FP8, adapter composition. ~800 LOC.
- **Total.** Roughly 3100 LOC. Estimated 14-18 implementation sessions.

This is lower-priority than CKVQ or CCR because (a) industry adoption is limited, (b) kernel maturity is the bottleneck even for those who try, and (c) NSL's target user may not benefit from 2:4 sparsity at typical inference scales. Worth staging but not critical.

§6 Gap 5 — Tokenizer/embedding/head-layer-specific quantization

§6.1 External state of the art

Embedding and output-head layers have fundamentally different statistics from transformer body layers:

Embedding layer. For a model with vocabulary size V and dimension d , the embedding matrix is $V \times d$. At $V = 128K$ and $d = 4096$, that is 524M parameters — often ~10% of a 7B model. Each embedding row is used when its token appears; row usage follows a Zipfian distribution (top 100 tokens dominate). This means most rows are rarely used but very sensitive when they are; a few rows dominate memory traffic.

Output head (LM head). The inverse projection $d \rightarrow V$. Same size. Hit pattern is different: during inference, every forward computes the full $d \rightarrow V$ projection regardless of which token will be chosen. At training time, the loss computation touches the full output; at inference time with sampling, it's the same.

Standard quantization approaches (e.g., GPTQ, AWQ) treat these layers specially. AWQ has explicit "first-and-last layer keep at FP16" heuristics. LLM.int8() treats the LM head as an outlier-sensitive layer. Most production quantization pipelines exempt these layers from the most aggressive precision.

Recent research (SmoothQuant, QuaRot, SpinQuant) uses rotation tricks (Hadamard or learned) to redistribute outlier magnitudes before quantization. These apply everywhere, including embedding and head layers, and alleviate the sensitivity issue somewhat.

Embedding-specific research is thinner. A few papers note that vocabulary-sized embedding matrices have low-rank structure that admits aggressive compression (EmbedQuant, etc.), but embedding compression has not reached production scale at the frontier.

Tokenizer-aware quantization is even less developed. The connection between the BPE merge table structure and the embedding matrix's statistics is recognized but not exploited in standard quantization pipelines. DIGS (Decomposition-Informed Gradient Sculpting), developed for CodeForge's TCFP, is a specific attack on this: the tokenizer's merge structure is known at compile time, so the output-head gradient can be sculpted to account for decomposition behavior.

§6.2 Why NSL has not addressed this

DIGS was developed for TCFP, which is the CodeForge project's quantization system — not NSL. The research paper exists (with a full 10-section write-up) but has not been ported to NSL's framework. NSL's CFTP and CFIP papers treat all layers uniformly (modulo WGGO per-layer decisions that are currently not sensitivity-aware).

§6.3 NSL-specific realization: CESH (Compiler-Exempt Sensitivity Handling)

CESH would provide a principled, compiler-visible treatment of embedding/head-layer specifics. Four design properties:

- (1) Layer-role annotation in the type system.** A layer's role (embedding, LM head, body) is a first-class property. `@weighted_layer(role="embedding")` is a user annotation, or it is inferred from the layer's position in the model (embedding is first, LM head is last). WGGO reads the role and applies role-specific policy.
- (2) Role-specific quantization policies.** Embedding layers default to FP16 unless an explicit low-precision annotation is applied. LM head layers default to FP16. Body layers default to CFTP's FP8+4-bit. This is simple policy but codifies industry practice in the compiler.
- (3) Port DIGS from TCFP to NSL.** DIGS's gradient-sculpting technique is NSL-adaptable: the W_{hi}/W_{lo} residual magnitude is exactly what CFTP already tracks (it is the 4-bit residual component). An NSL-DIGS would read the residual magnitude at each backward step, construct the confidence mask, and attenuate the output-head gradient before the optimizer step. The implementation is small (~200 LOC) and the integration points with CFTP are clean.
- (4) Embedding low-rank decomposition.** An optional technique: compress the embedding matrix via low-rank factorization ($r \ll V$), computed at training time or inference-load time. Compose with WRGA's existing low-rank machinery. This is an optional enhancement, not a default.

§6.4 Implementation scope

- **Layer role annotation + inference.** Semantic stage identifies embedding/head layers. ~200 LOC.
- **Role-aware WGGO policy.** Extend the decision table with a role-dependent default. ~150 LOC.
- **DIGS port.** Implement the gradient-sculpting mask calculation, hook into backward. ~300 LOC.
- **Low-rank embedding (optional).** Factor the embedding matrix; produce the factored forward path. ~400 LOC.
- **Testing.** Per-layer accuracy measurements, DIGS gradient-equivalence tests, embedding compression accuracy-retention. ~500 LOC.
- **Total.** Roughly 1550 LOC. Estimated 6-10 implementation sessions.

This is the smallest of the five gaps in scope. It is mostly about making existing industry practice a first-class NSL feature rather than an undocumented convention.

§7 Prioritization and sequencing

The five gaps have different urgency and different dependencies. The following prioritization reflects both factors.

Tier 1 — Next research priority.

CCR (Compiler-Chosen Recomputation). This is the highest-impact missing technique. Activation memory is the single biggest training memory consumer, and NSL has no first-class answer. The MILP-based approach compositely handles save-vs-compress-vs-recompute with existing NSL infrastructure (WGGO, CSHA, CFIP, CPFT). Estimated 10-14 sessions. No new external dependencies.

CKVQ (Compiler-Known Value Quantization). The second priority. KV cache dominates long-context inference memory; every major inference framework has this. NSL's type-system and PCA-paged-allocation advantages make the NSL-specific realization meaningfully better than a PyTorch port. Estimated 8-12 sessions. Dependencies on PCA, CFA, CFIP (all shipped).

Tier 2 — Stage after Tier 1.

CESH (Compiler-Exempt Sensitivity Handling). The smallest-scope technique. Much of its value is documentation of existing industry practice as a first-class NSL concept. Porting DIGS is mostly done in the TCFP paper; translating to NSL is ~300 LOC. Estimated 6-10 sessions.

CGAC (Compiler-Generated All-Reduce Compression). Lower priority because NSL has not yet been validated at multi-node scale. CGAC's value requires that scale. Once NSL ships multi-node training, CGAC becomes Tier 1. Estimated 12-18 sessions. Dependency on a multi-node test environment (currently absent).

Tier 3 — Stage last.

CSS (Compiler-Scheduled Sparsity). Lower priority because industry adoption is limited and kernel maturity is the bottleneck. CSS's theoretical 2× throughput is not yet a production reality. Worth having for completeness and for users who specifically target Blackwell with 4:8 sparse NVFP4, but not a compelling next-priority. Estimated 14-18 sessions.

The recommended sequence: CCR → CKVQ → CESH → CGAC (after multi-node) → CSS (last).

A summary table:

Technique	Domain	Scope (LOC)	Sessions	Priority	Dependencies
CCR	Activation recomputation	~2800	10-14	Tier 1	CSHA, CFIP, WGGO, M36, CPFT (all shipped)
CKVQ	KV cache quantization	~1900	8-12	Tier 1	PCA, CFA, CFIP, M35 (all shipped)
CESH	Embedding/head quantization	~1550	6-10	Tier 2	CFTP (shipped)
CGAC	All-reduce compression	~3200	12-18	Tier 2	Multi-node test environment (absent)
CSS	Structured sparsity	~3100	14-18	Tier 3	Sparse type scaffolding (partial)

All five techniques are implementable on current NSL; none requires a rewrite of any existing technique. Several require non-trivial composition with existing techniques but that is the point of the composition paper — those compositions are well-understood.

§8 Conclusion

The fourteen shipped NSL technique papers are comprehensive within their declared scope (training-time precision, attention, adapters, memory, distributed). Five technique domains lie outside that scope: activation recomputation, KV cache quantization, gradient compression, structured sparsity, and role-specific embedding/head quantization. For each, external research has established standard approaches; for each, NSL has an opportunity to provide a meaningfully better realization by leveraging its type system and compile-time analyses.

The five techniques sum to approximately 12,550 lines of code and 50–70 implementation sessions. They can be staged over roughly six months of focused work. The end state is an NSL compiler that addresses every major technique domain in modern ML systems — training precision, inference precision, attention, adapters, memory (both gradient and activation), distributed (both sharding and compression), sparsity, and role-specific specialization.

The next paper in this series addresses the unexploited cross-paper connections and individual paper refinements surfaced by the same April 2026 audit.

End of paper.